

MAESTRÍA EN INTELIGENCIA ARTIFICIAL – UNIR

Procesamiento del Lenguaje Natural y Machine Learning para Texto

Preparación de Texto y Cómputo de Similitud

Guía de estudio completa con ejemplos y ejercicios resueltos

Pablo Alberto Duque Marín

2026

Tabla de Contenidos

1	Introducción y contexto	3
2	Extracción y tokenización del texto crudo	4
2.1	Codificación de caracteres	4
2.2	Tokenización: conceptos y retos	5
2.3	Consideraciones especiales para la Web	6
3	De tokens a términos	7
3.1	Eliminación de palabras vacías (stop words)	7
3.2	Guiones (hyphens)	8
3.3	Normalización de mayúsculas	8
3.4	Consolidación por uso	9
3.5	Stemming y lematización	9
4	Representación vectorial y normalización TF-IDF	11
4.1	Modelo binario (Bernoulli)	11
4.2	Modelo TF-IDF	12
5	Cómputo de similitud entre documentos	14
5.1	Distancia euclidiana y sus limitaciones	14
5.2	Similitud coseno	15
5.3	Coeficiente de Jaccard	17
5.4	Comparación de métricas	18
6	¿Cuándo conviene normalizar? Análisis crítico	19
7	Ejercicios resueltos paso a paso	20
8	Resumen y mapa conceptual	24

1. Introducción y contexto

El texto es uno de los tipos de datos más abundantes en el mundo digital: correos, redes sociales, artículos científicos, páginas web, reportes médicos y contratos legales son todos texto. Sin embargo, a diferencia de los datos numéricos estructurados, el texto es inherentemente **desestructurado**: no viene en columnas ordenadas ni en escalas predefinidas. Antes de poder aplicar cualquier algoritmo de aprendizaje automático, es imprescindible transformar ese texto en una **representación matemática** que los modelos puedan procesar.

Este proceso de transformación se llama **preprocesamiento de texto** y consta de varias etapas encadenadas. Cada etapa tiene un propósito específico y sus propios desafíos. En este documento estudiaremos cada una en detalle, con ejemplos concretos en español e inglés, y posteriormente veremos cómo, una vez representado el texto como vectores numéricos, podemos medir la similitud entre documentos.

■ Objetivo central

El objetivo final del preprocesamiento es convertir texto arbitrario en vectores numéricos de dimensión fija, de modo que documentos con contenido similar queden representados por vectores cercanos en el espacio matemático, y documentos distintos queden alejados.

El pipeline completo de preprocesamiento sigue este orden:

Paso	Nombre	¿Qué hace?
1	Extracción y parsing	Convierte el formato original (HTML, PDF, Word) en texto plano
2	Tokenización	Divide la secuencia de caracteres en unidades llamadas tokens
3	Eliminación de stop words	Descarta palabras muy frecuentes y poco discriminativas
4	Stemming / Lematización	Reduce las palabras a su raíz o forma canónica
5	Representación vectorial	Construye la matriz documento-término con pesos TF-IDF
6	Cómputo de similitud	Calcula distancias o similitudes entre vectores de documentos

2. Extracción y tokenización del texto crudo

2.1 Codificación de caracteres

El primer reto es que el texto no existe en el computador como tal: existe como una secuencia de bytes, y para interpretarlos como caracteres se necesita saber qué **esquema de codificación** se usó.

El estándar más usado hoy es **Unicode**, un sistema que asigna un identificador único a cada carácter de prácticamente todos los idiomas del mundo, incluyendo símbolos matemáticos y caracteres históricos. Las implementaciones más comunes de Unicode son:

- **UTF-8**: Usa entre 1 y 4 bytes por carácter. Compatible con ASCII. Dominante en la web occidental y en la mayoría de sistemas Linux/Mac.
- **UTF-16**: Usa 2 bytes fijos. Común en idiomas asiáticos (japonés, chino, coreano) y en el sistema interno de Windows y Java.
- **ASCII**: Sólo 128 caracteres (letras latinas, dígitos, puntuación básica). Subconjunto de UTF-8. Insuficiente para idiomas no latinos.

■ ■ Advertencia práctica

En la práctica, cuando construyas un pipeline de PLN, asegúrate siempre de leer archivos con `encoding='utf-8'` en Python. Si el archivo usa otra codificación (latin-1, cp1252, etc.) y la mezclas con UTF-8, obtendrás errores o caracteres corruptos que contaminan todo el análisis posterior.

2.2 Tokenización: conceptos y retos

■ Definición: Token

Un **token** es una secuencia contigua de caracteres con significado semántico que se trata como una unidad indivisible durante el procesamiento. Intuitivamente, corresponde a una 'palabra', aunque veremos que la frontera entre palabras no siempre es clara.

La regla más simple de tokenización es: separar por espacios en blanco y eliminar signos de puntuación. Pero esta regla falla en decenas de casos reales. Veamos los más importantes:

Casos problemáticos de tokenización:

Caso	Ejemplo	Problema	Solución habitual
Nombres propios compuestos	"Puentes", "Nueva York"	Separar pierde el significado	Diccionarios de frases o n-gramas
Números decimales	"3.14", "2,5 kg"	El punto/coma no es separador	Reglas: punto entre dígitos → no separar
Fechas y horas	"06/20/2003", "8:20 PM"	"/" y ":" no son separadores	Reconocimiento de patrones numéricos
Acrónimos	"Dr.", "M.D.", "EE.UU."	El punto no termina oración	Lista de acrónimos almacenada

Apóstrofes	"don't", "o'clock"	¿Separar o mantener unido?	Reglas: apóstrofe al inicio/fin → eliminar
Guiones	"state-of-the-art"	¿Una o tres palabras?	Diccionario de palabras compuestas

—■ Ejemplo de tokenización

Considera la oración: «Después de dormir 4 h, decidió volver a dormir otras dos.»

Los tokens resultantes serían: { 'Después', 'de', 'dormir', '4', 'h', 'decidió', 'volver', 'a', 'dormir', 'otras', 'dos' }. Nótese que 'dormir' aparece dos veces (tokens distintos), '4' y 'h' son tokens separados porque 'h' es una abreviación de 'horas', y la coma y el punto final se descartan. El preprocesamiento posterior consolidará los dos 'dormir' en un único término con frecuencia 2.

2.3 Consideraciones especiales para documentos web

Las páginas HTML son la fuente de texto más común en aplicaciones reales de PLN. Presentan retos específicos que no aparecen en texto plano:

- **Etiquetas HTML:** Todo el marcado (div, p, a href, etc.) debe eliminarse antes de tokenizar. Herramientas como BeautifulSoup en Python facilitan esta limpieza.
- **Texto ancla (anchor text):** El texto visible de los hipervínculos es frecuentemente una descripción de la página destino, no del contenido actual. Según el contexto, puede eliminarse o añadirse al documento destino como metadato útil.
- **Bloques irrelevantes:** Anuncios, disclaimers, menús de navegación y pies de página contaminan el texto principal. Identificar el bloque principal es en sí mismo un problema de minería de datos, resoluble por clasificación supervisada o emparejamiento de plantillas HTML.
- **Campos con distinto peso:** El título de una página web tiene más peso semántico que el cuerpo. Los sistemas de recuperación de información suelen ponderar el campo title más que el campo body.

3. De tokens a términos

Una vez obtenidos los tokens, es necesario transformarlos en **términos**: unidades normalizadas que formarán las dimensiones de nuestro espacio vectorial. Esta fase incluye varias subtécnicas que se aplican en secuencia.

3.1 Eliminación de palabras vacías (stop words)

Las palabras vacías son palabras de uso muy frecuente en el idioma que aportan escasa información discriminativa para las tareas de minería. Ejemplos en español: *el, la, los, las, de, que, y, en, a, un, una*. En inglés: *{i('the, a, an, is, was, for, of, to, in, and')}*.

El criterio de identificación puede ser:

- Listas predefinidas por idioma (las más comunes y fiables).
- Umbral de frecuencia: si una palabra aparece en más del X% de los documentos, se elimina.
- IDF bajo: palabras con frecuencia de documento alta tienen IDF cercano a cero (veremos IDF en la sección 4).

■ ■ Cuidado con la eliminación agresiva

Eliminar stop words es la versión **dura** de una estrategia más suave: simplemente **bajarles el peso** con la normalización IDF. Algunos sistemas modernos (especialmente motores de búsqueda) prefieren no eliminarlas del todo porque frases como 'to be or not to be' o 'el señor de los anillos' perderían su significado si se eliminan las stop words.

3.2 Tratamiento de guiones

Los guiones son ambiguos: pueden unir palabras en un compuesto semántico único (*state-of-the-art*) o simplemente conectar un modificador con su sustantivo (*five-year-old girl*). La regla por defecto es mantener el término guionado como una unidad, pues separarlo suele alterar el significado. Sin embargo, existen diccionarios de palabras guionadas comunes que permiten tomar decisiones más precisas caso a caso.

3.3 Normalización de mayúsculas (case folding)

En general se convierte todo a minúsculas para evitar tratar 'Python' y 'python' como términos distintos. Sin embargo, la capitalización a veces es semánticamente significativa: 'Bob' (nombre propio) vs 'bob' (verbo en inglés). El proceso de determinar la capitalización correcta de cada token se llama **truecasing** y es en sí mismo un problema de aprendizaje automático. La heurística simplificada más común es: convertir a minúsculas todo lo que esté al inicio de una oración o en títulos; conservar el resto.

3.4 Consolidación por uso (usage-based consolidation)

Distintos escritores pueden usar variantes del mismo término: *color* vs *colour*, *naïve* vs *naïve*, *e-mail* vs *email*. Estas variantes deben consolidarse en un único término estándar. La implementación típica usa una tabla hash que mapea cada variante conocida a su forma canónica.

3.5 Stemming y lematización

Quizás la técnica más importante de normalización de términos. El objetivo es consolidar formas flexionadas de una misma palabra en su raíz común.

■ Stemming

Stemming: proceso de extraer la raíz morfológica de una palabra mediante heurísticas. El resultado no es necesariamente una palabra válida del idioma, sino un identificador de raíz. Ejemplo: 'eating', 'eaten', 'eats' → 'eat'. El algoritmo clásico es el **Porter Stemmer** (también conocido como Snowball).

■ Lematización

Lematización: proceso más sofisticado que determina la forma canónica (lema) de la palabra usando su parte del discurso. Ejemplo: 'ate' → 'eat' (correcto), no 'at' (que sería el resultado de un stemmer agresivo). Requiere un lexicon y conocimiento gramatical del idioma.

Técnicas de stemming

- **Tablas de búsqueda semi-automáticas:** Se construye una tabla que mapea cada variante conocida a su raíz. Preciso pero difícil de mantener para vocabularios grandes. — *Ejemplo:* 'eating' → buscar en tabla → 'eat'
- **Eliminación de sufijos (suffix stripping):** Se almacena una lista de reglas: eliminar '-ing', '-ed', '-ly', '-tion', etc. Simple y rápido, pero comete errores ('hoping' → 'hop'). — *Ejemplo:* 'hoping' → eliminar '-ing' → 'hop' (incorrecto)
- **Lematización con POS tagging:** Determina la parte del discurso (verbo, sustantivo, adjetivo) y aplica reglas morfológicas específicas para cada categoría. — *Ejemplo:* 'ate' + POS=VERB → lema 'eat'

—■ Ejemplo completo del pipeline tokens → términos

Considera la oración: 'Los estudiantes están estudiando estadística aplicada.'

Después del pipeline completo (minúsculas → stop words → stemming):

Tokens originales: [los, estudiantes, están, estudiando, estadística, aplicada]

Tras eliminar stop words: [estudiantes, están, estudiando, estadística, aplicada]

Tras stemming (español): [estudi, estudi, estudi, estadíst, aplic]

Términos finales únicos con frecuencias: { estudi:3, estadíst:1, aplic:1 }

4. Representación vectorial y normalización

TF-IDF

Una vez extraídos y normalizados los términos, se construye una **representación numérica** del texto. El modelo estándar es el **espacio vectorial**: cada documento se representa como un vector en un espacio de dimensión d , donde d es el tamaño del vocabulario (número de términos únicos en la colección). La representación es extremadamente escasa (sparse): un documento típico contiene sólo unos cientos de términos distintos, mientras el vocabulario puede tener cientos de miles.

■ Matriz documento-término

La matriz documento-término D es de tamaño $n \times d$, donde n es el número de documentos y d el número de términos únicos. La entrada $D[i][j]$ indica la importancia del término j en el documento i . Como n y d pueden ser muy grandes pero la mayoría de entradas son cero, en la práctica siempre se almacena en formato disperso (sparse matrix).

4.1 Modelo binario (Bernoulli / boolean)

En el modelo binario, la entrada $D[i][j] = 1$ si el término j aparece en el documento i , y 0 en caso contrario. No se consideran las frecuencias.

Ventajas: compacto, compatible con algoritmos de minería de patrones frecuentes y con el clasificador de Bayes Bernoulli.

Desventajas: pierde información de frecuencia (un documento donde 'machine' aparece 50 veces se trata igual que uno donde aparece 1 vez).

Ejemplo de matriz binaria:

	gato	perro	come	carne	duerme
Doc 1: El gato come carne	1	0	1	1	0
Doc 2: El perro duerme	0	1	0	0	1
Doc 3: El gato duerme	1	0	0	0	1

4.2 Modelo TF-IDF

El modelo TF-IDF (Term Frequency – Inverse Document Frequency) es el estándar en casi todas las aplicaciones de recuperación de información y minería de texto. Combina dos factores: la frecuencia del término en el documento y la rareza del término en la colección completa.

Factor 1 – Frecuencia del término (TF)

La frecuencia cruda x_i de un término es el número de veces que aparece en el documento. Como documentos largos tendrán frecuencias crudas mayores por defecto, se aplican funciones de amortiguación para reducir el efecto de repetición:

$$\text{TF amortiguado} = \sqrt{x_i} \text{ o } \text{TF amortiguado} = \log(1 + x_i)$$

Factor 2 – Frecuencia inversa de documento (IDF)

Si un término aparece en muchos documentos, es poco discriminativo (podría ser una stop word no eliminada). El IDF mide la rareza del término en la colección:

$$\text{IDF}_i = \log(n / n_i)$$

donde n es el número total de documentos y n_i es el número de documentos en los que aparece el término i . Si un término aparece en **todos** los documentos, $\text{IDF} = \log(1) = 0$. Si aparece en muy pocos, IDF es grande.

Peso TF-IDF final

$$w_{ij} = x_{ij} * \text{IDF}_j = x_{ij} * \log(n / n_j)$$

■ Ejemplo numérico de TF-IDF

Colección de 3 documentos. El término 'gato' aparece en 2 de ellos. El término 'estadística' aparece en 1.

$$\text{IDF}(\text{'gato'}) = \log(3/2) = \log(1.5) \approx 0.405$$

$$\text{IDF}(\text{'estadística'}) = \log(3/1) = \log(3) \approx 1.099$$

Si en el Doc 1, 'gato' aparece 3 veces y 'estadística' aparece 2 veces:

$$w(\text{'gato'}, \text{Doc1}) = 3 \times 0.405 = 1.215$$

$$w(\text{'estadística'}, \text{Doc1}) = 2 \times 1.099 = 2.197$$

El peso de 'estadística' es mayor porque es un término más raro y específico.

5. Cómputo de similitud entre documentos

Representados los documentos como vectores numéricos, necesitamos una forma de medir qué tan parecidos son dos documentos. Esto es fundamental para tareas como búsqueda de información, agrupamiento (clustering) y clasificación.

5.1 Distancia euclidiana y sus limitaciones

La primera idea natural es usar la distancia euclidiana, ya conocida de la geometría:

$$\text{dist_euclid}(X, Y) = \sqrt{\sum_i (x_i - y_i)^2}$$

El problema: la distancia euclidiana es muy sensible a la longitud de los documentos. Un documento largo y uno corto que traten exactamente el mismo tema tendrán una distancia euclidiana grande simplemente porque uno tiene muchos más términos.

—■ Problema de la distancia euclidiana con texto

Considera cuatro oraciones:

A: 'Ella se sentó.' (3 palabras)

B: 'Ella bebió café.' (3 palabras)

C: 'Ella dedicó mucho tiempo a aprender minería de texto.' (9 palabras)

D: 'Ella invirtió esfuerzo significativo en aprender minería de texto.' (9 palabras)

A y B son semánticamente distintas pero cortas $\rightarrow \text{dist_euclid} = \sqrt{4} = 2$

C y D son semánticamente similares pero largas $\rightarrow \text{dist_euclid} = \sqrt{6} > 2$

¡La distancia euclidiana dice que A-B son más similares que C-D, lo cual es incorrecto!

Este es el problema de la variación de longitud de documentos.

5.2 Similitud coseno

La solución estándar es medir el ángulo entre los vectores, no su distancia. El coseno del ángulo entre dos vectores no depende de su longitud (módulo), sino únicamente de su dirección. Dos documentos que tratan el mismo tema apuntarán en direcciones similares independientemente de cuántas palabras tengan.

$$\cos(X, Y) = \left(\sum_i x_i * y_i \right) / \left(\sqrt{\sum_i x_i^2} * \sqrt{\sum_i y_i^2} \right)$$

El coseno siempre está en el rango **[0, 1]** para vectores de texto (porque todas las entradas son no negativas). Un coseno de 1 indica documentos idénticos en contenido; un coseno de 0 indica documentos sin ningún término en común.

Interpretación en el caso binario:

Cuando X e Y son vectores binarios, sean S_x y S_y los conjuntos de palabras presentes en cada documento:

$$\cos(S_x, S_y) = |S_x \cap S_y| / \sqrt{|S_x| * |S_y|}$$

Es decir, el coseno es la media geométrica de las fracciones de términos compartidos en cada documento. Si el 80% de las palabras del doc A aparecen en el doc B, y el 60% de las palabras de B aparecen en A, entonces $\cos \approx \sqrt{0.80 \times 0.60} \approx 0.69$.

—■ Cálculo numérico de la similitud coseno

Sean $X = (2, 3, 0, 5, 0)$ e $Y = (0, 1, 2, 2, 0)$.

Producto punto: $(2 \times 0) + (3 \times 1) + (0 \times 2) + (5 \times 2) + (0 \times 0) = 0 + 3 + 0 + 10 + 0 = 13$

$\|X\| = \sqrt{2^2 + 3^2 + 0^2 + 5^2 + 0^2} = \sqrt{4+9+0+25+0} = \sqrt{38} \approx 6.164$

$\|Y\| = \sqrt{0^2 + 1^2 + 2^2 + 2^2 + 0^2} = \sqrt{0+1+4+4+0} = \sqrt{9} = 3$

$\cos(X, Y) = 13 / (6.164 \times 3) = 13 / 18.49 \approx 0.703$

Interpretación: X e Y comparten aproximadamente el 70.3% de su contenido (en términos ponderados), lo cual indica alta similitud semántica.

Relación entre coseno y distancia euclidiana normalizada:

Si normalizamos cada vector a norma unitaria ($\|X\| = 1$) antes de computar, entonces la distancia euclidiana y el coseno son equivalentes:

$$\|X - Y\|^2 = \|X\|^2 + \|Y\|^2 - 2(X \cdot Y) = 1 + 1 - 2 \cdot \cos(X, Y) = 2(1 - \cos(X, Y))$$

Esto significa que si normalizamos los vectores con anticipación, podemos usar cualquiera de los tres: distancia euclidiana, producto punto, o similitud coseno, y obtendremos los mismos resultados en clasificación y recuperación.

5.3 Coeficiente de Jaccard

El coeficiente de Jaccard es especialmente útil cuando se usa la representación binaria. Mide la proporción de términos compartidos respecto al total de términos distintos en la unión:

$$\text{Jaccard}(S_x, S_y) = |S_x \cap S_y| / |S_x \cup S_y|$$

También puede generalizarse para representaciones TF-IDF:

$$\text{Jaccard}(X, Y) = (\sum x_i y_i) / (\sum x_i^2 + \sum y_i^2 - \sum x_i y_i)$$

—■ Ejemplo: Jaccard vs Coseno

Doc A = {gato, come, carne} → S_A = {gato, come, carne}

Doc B = {gato, duerme} → S_B = {gato, duerme}

Intersección: {gato} → $|S_A \cap S_B| = 1$

Unión: {gato, come, carne, duerme} → $|S_A \cup S_B| = 4$

Jaccard(A, B) = $1/4 = 0.25$

Coseno(A, B) = $1 / (\sqrt{3} \times \sqrt{2}) = 1/\sqrt{6} \approx 0.408$

En general: Jaccard ≤ Coseno (el coseno es siempre mayor o igual).

5.4 Comparación de métricas

Métrica	Fórmula clave	Rango	Cuándo usarla
Distancia euclidiana	$\sqrt{\sum (x_i - y_i)^2}$	$[0, \infty)$	Sólo si los vectores están normalizados a norma unitaria
Similitud coseno	$X \cdot Y / (\ X\ \cdot \ Y\)$	$[0, 1]$	Caso general; representación TF-IDF; documentos de longitudes variables
Coefficiente de Jaccard	$ A \cap B / A \cup B $	$[0, 1]$	Representación binaria; comparación de conjuntos de características
Coefficiente de Pearson	Covarianza normalizada	$[-1, 1]$	Cuando se quiere capturar correlación lineal entre perfiles de términos

6. ¿Cuándo conviene normalizar? Análisis crítico

Una pregunta importante que los practicantes suelen ignorar: **¿la normalización TF-IDF y el stemming siempre mejoran los resultados?** La respuesta corta es: no necesariamente. Depende fuertemente de la tarea.

Técnica	Contexto donde ayuda	Contexto donde puede perjudicar
Normalización TF-IDF	Recuperación de información (búsqueda web): el uso relativo de los términos ayuda a encontrar los documentos más relevantes.	Clustering de documentos y modelos probabilísticos (LDA): las frecuencias relativas de los términos pueden ser irrelevantes.
Stemming	Consultas de búsqueda cortas (pocos términos): ayuda a encontrar documentos relevantes que no coinciden exactamente con la consulta.	Declaración de la tarea de clasificación: stemming puede degradar el rendimiento.
Eliminación de stopwords	Texto general: reduce ruido y dimensión.	Análisis de sentimiento y frases fijas: 'not good' pierde 'not' y cambia el significado.

■ ■ Principio de diseño de pipelines de PLN

El mensaje central es que estas técnicas son herramientas heredadas de la recuperación de información tradicional. En el contexto de machine learning moderno, las restricciones del problema son distintas y no siempre aplican las mismas reglas. Siempre se debe experimentar con y sin normalización, medir el impacto en la métrica de evaluación de la tarea concreta, y decidir empíricamente.

7. Ejercicios resueltos paso a paso

Ejercicio 1 – Tokenización y eliminación de stop words

■ Enunciado

Oración: «Después de dormir 4 horas, él decidió volver a dormir otras dos más.»

Stop words del español (lista parcial): de, él, a, más, otras, después

■ Solución

Paso 1 – Tokenización (separar por espacios, eliminar puntuación):

[Después, de, dormir, 4, horas, él, decidió, volver, a, dormir, otras, dos, más]

Paso 2 – Minúsculas:

[después, de, dormir, 4, horas, él, decidió, volver, a, dormir, otras, dos, más]

Paso 3 – Eliminar stop words {de, él, a, más, otras, después):

[dormir, 4, horas, decidió, volver, dormir, dos]

Paso 4 – Stemming simplificado:

dormir→dorm, horas→hora, decidió→decid, volver→volv, dos→dos, 4→4

Resultado: [dorm, 4, hora, decid, volv, dorm, dos]

Paso 5 – Representación vectorial (términos únicos con frecuencias):

{ dorm:2, 4:1, hora:1, decid:1, volv:1, dos:1 }

Ejercicio 2 – Cálculo de IDF y pesos TF-IDF

■ Enunciado

Colección de $n=4$ documentos. Los términos tienen las siguientes frecuencias de documento:

'machine': aparece en 4 documentos $\rightarrow n_i = 4$

'learning': aparece en 3 documentos $\rightarrow n_i = 3$

'neural': aparece en 1 documento $\rightarrow n_i = 1$

En el documento D1: 'machine' aparece 5 veces, 'learning' 3 veces, 'neural' 2 veces.

Calcula los pesos TF-IDF de cada término en D1.

■ Solución

$IDF('machine') = \log(4/4) = \log(1.000) = 0.000$

$IDF('learning') = \log(4/3) = \log(1.333) \approx 0.288$

$IDF('neural') = \log(4/1) = \log(4.000) \approx 1.386$

TF-IDF en D1 (sin amortiguación):

$w('machine', D1) = 5 \times 0.000 = 0.000 \leftarrow$ aparece en TODOS los docs, peso nulo

$w('learning', D1) = 3 \times 0.288 = 0.864$

$w('neural', D1) = 2 \times 1.386 = 2.772 \leftarrow$ raro en la colección, peso alto

Conclusión: 'machine' recibe peso cero porque no discrimina entre documentos. 'neural' recibe el mayor peso por ser el más específico de la colección.

Ejercicio 3 – Similitud coseno entre dos documentos

■ Enunciado

Vocabulario = {gato, perro, come, duerme, carne}

Doc A: 'El gato come carne y come más' → $X = (1, 0, 2, 0, 1)$

Doc B: 'El perro come y duerme' → $Y = (0, 1, 1, 1, 0)$

Calcula la similitud coseno entre A y B.

■ Solución

$X = (1, 0, 2, 0, 1)$ $Y = (0, 1, 1, 1, 0)$

Paso 1 – Producto punto $X \cdot Y$:

$$(1 \times 0) + (0 \times 1) + (2 \times 1) + (0 \times 1) + (1 \times 0) = 0 + 0 + 2 + 0 + 0 = 2$$

Paso 2 – Norma de X:

$$\|X\| = \sqrt{1^2 + 0^2 + 2^2 + 0^2 + 1^2} = \sqrt{1+0+4+0+1} = \sqrt{6} \approx 2.449$$

Paso 3 – Norma de Y:

$$\|Y\| = \sqrt{0^2 + 1^2 + 1^2 + 1^2 + 0^2} = \sqrt{0+1+1+1+0} = \sqrt{3} \approx 1.732$$

Paso 4 – Similitud coseno:

$$\cos(X, Y) = 2 / (2.449 \times 1.732) = 2 / 4.243 \approx 0.471$$

Interpretación: A y B comparten aproximadamente el 47.1% de su contenido ponderado.

Similitud moderada, tiene sentido porque sólo comparten 'come'.

Ejercicio 4 – Coeficiente de Jaccard (binario)

■ Enunciado

Usando los mismos documentos del ejercicio 3 (representación binaria):

$S_A = \{\text{gato, come, carne}\}$ (los 3 términos con valor 1 o más)

$S_B = \{\text{perro, come, duerme}\}$

Calcula el coeficiente de Jaccard y compáralo con el coseno.

■ Solución

$S_A = \{\text{gato, come, carne}\}$

$S_B = \{\text{perro, come, duerme}\}$

Intersección: $S_A \cap S_B = \{\text{come}\} \rightarrow |\text{intersección}| = 1$

Unión: $S_A \cup S_B = \{\text{gato, come, carne, perro, duerme}\} \rightarrow |\text{unión}| = 5$

$$\text{Jaccard}(A, B) = 1 / 5 = 0.200$$

Coseno binario:

$X_{\text{bin}} = (1, 0, 1, 0, 1)$ $Y_{\text{bin}} = (0, 1, 1, 1, 0)$

$$X \cdot Y = 1, \|X\| = \sqrt{3}, \|Y\| = \sqrt{3}$$

$$\cos = 1/3 \approx 0.333$$

Verificación: $\text{Jaccard}(0.200) \leq \text{Coseno}(0.333)$ ✓

El coseno siempre es mayor o igual al Jaccard para vectores binarios.

8. Resumen y mapa conceptual

Concepto	Definición clave	Dato importante
Token	Unidad indivisible de texto extraída por parsing	por término; aún no normalizado
Stop word	Palabra frecuente y poco discriminativa	Eliminar es variante 'dura' de IDF
Stemming	Reducción a raíz morfológica con heurística	Puede cambiar semántica ('hoping'→'hop')
Lematización	Reducción a forma canónica usando POS	Más preciso pero más lento que stemming
Modelo binario	Presencia/ausencia de término (0 o 1)	Útil para Naive Bayes Bernoulli y Jaccard
TF	Frecuencia del término en el documento	Se amortigua con log o sqrt
IDF	$\log(n / n_i)$; rareza del término en la colección	IDF=0 si el término está en todos los docs
TF-IDF	TF × IDF; peso ponderado del término	Estándar en recuperación de información
Similitud coseno	Ángulo entre vectores; normaliza por longitud	Rango [0,1]; recomendada para texto
Jaccard	Intersección / Unión de términos	Jaccard ≤ Coseno siempre
Dist. euclidiana	Distancia geométrica entre vectores	Problemática sin normalizar por longitud

Pipeline completo resumido:

Etapas	Entrada	Salida	Técnicas clave
1. Extracción	HTML / PDF / Word	Texto plano UTF-8	Parsing HTML, extracción de bloques
2. Tokenización	Secuencia de caracteres	Lista de tokens	Reglas de separación, manejo de hifens, URLs
3. Normalización de tokens	Tokens crudos	Tokens normalizados	Minúsculas, stop words, consolidación por uso
4. Stemming / Lematización	Tokens normalizados	Términos raíz	Porter Stemmer, lematizador con POS
5. Vectorización	Lista de términos	Vector TF-IDF	Matriz documento-término, IDF, amortiguación
6. Similitud	Par de vectores	Valor en [0,1]	Coseno, Jaccard, Euclidiana normalizada

■ Puntos clave para recordar

- El preprocesamiento transforma texto desestructurado en vectores numéricos.
- La tokenización es específica del idioma y requiere reglas cuidadosas.
- El stemming y la lematización consolidan variantes morfológicas en términos únicos.
- TF-IDF pondera los términos por su frecuencia local y rareza global.
- La similitud coseno es la métrica estándar para texto por su invarianza a la longitud.
- Jaccard $\leq$ Coseno siempre para representaciones binarias.
- No toda normalización mejora todos los modelos: experimentar es esencial.